

线程同步概述

北京理工大学计算机学院
金旭亮



线程同步机制



线程同步机制是一套用于协调线程间的数据访问（Data Access）以及相互协作（cooperation）的机制，它就好比是车辆上路需要遵循的交通规则。

一个简单的多线程示例

自定义一个线程类：

```
2 usages
class MyThread extends Thread {
    2 usages
    public MyThread(String name) {
        super(name);
    }
    @Override
    public void run() {
        for (int i = 0; i < 10; i++)
            System.out.println(getName() + "处理" + i);
    }
}
```

启动两个线程并发运行：

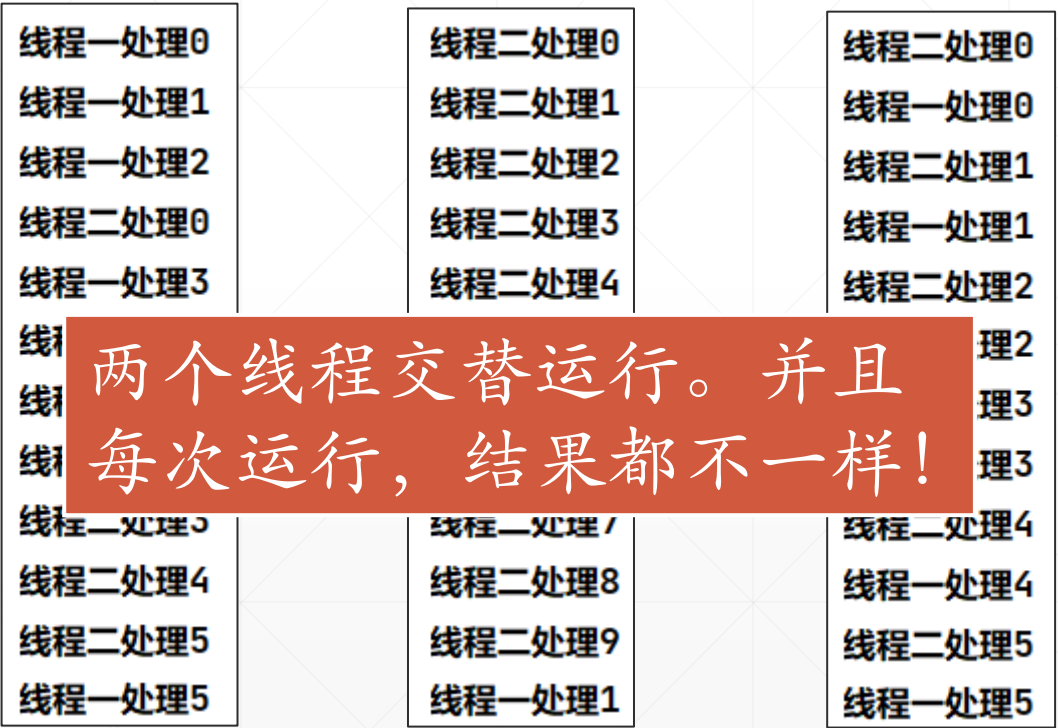
```
public class TwoThreadTest {
    public static void main(String[] args) {
        new MyThread(name: "线程一").start();
        new MyThread(name: "线程二").start();
    }
}
```



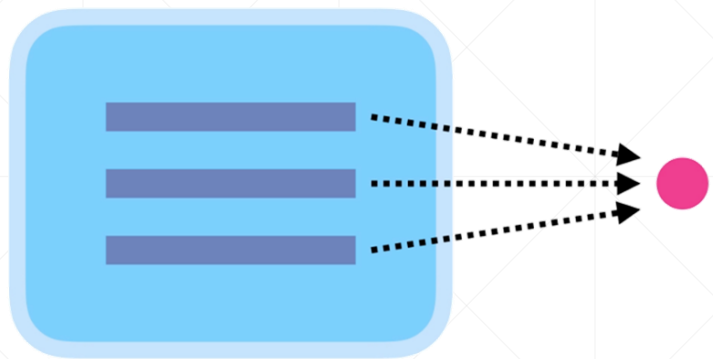
只有调用Thread类的Start方法，才能真正地在一个独立的线程中执行代码，直接调用Thread类的run方法，并不能启动一个新的线程，此时，代码是在调用者线程中执行的。

多线程程序执行结果的不确定性

TwoThreadTest示例中的两个线程同时运行相同的代码，但



多线程访问共享数据示例-1



此类封装了线程函数，它向
SimpleArray中写入数据

c • SimpleArray	
m	SimpleArray(int)
m	toString() String
m	add(int) void

多线程共享的数据对象，内部封装了一个数组

c • ArrayWriter	
m	ArrayWriter(int, SimpleArray)
m	run() void

多线程访问共享数据示例-2

```
public class SharedArrayTest {  
    public static void main(String[] arg) throws InterruptedException {  
        //创建一个可保存6个整数的SimpleArray对象  
        SimpleArray sharedSimpleArray = new SimpleArray(size: 6);  
        //创建两个工作线程向SimpleArray写入数据  
        var thread1 = new Thread(new ArrayWriter(value: 1, sharedSimpleArray));  
        var thread2 = new Thread(new ArrayWriter(value: 11, sharedSimpleArray));  
        //启动两个工作线程  
        thread1.start();  
        thread2.start();  
        //等待两个工作线程结束  
        thread1.join();  
        thread2.join();  
        System.out.println("所有线程运行结束, 最终结果为: ");  
        System.out.println(sharedSimpleArray);  
    }  
}
```

运行截图

```
Run: SharedArrayTest x
"C:\Program Files\Java\jdk-17\bin\java.exe"
Thread-1 将数值 11 写入到了数组的第 0 个元素。
下一个可写入的位置是: 1
Thread-0 将数值 1 写入到了数组的第 0 个元素。
下一个可写入的位置是: 2
Thread-1 将数值 12 写入到了数组的第 1 个元素。
下一个可写入的位置是: 3
Thread-0 将数值 2 写入到了数组的第 2 个元素。
下一个可写入的位置是: 4
Thread-1 将数值 13 写入到了数组的第 3 个元素。
下一个可写入的位置是: 5
Thread-0 将数值 3 写入到了数组的第 4 个元素。
下一个可写入的位置是: 6
所有线程运行结束, 最终结果为:

Contents of SimpleArray:
[1, 12, 2, 13, 3, 0]

Process finished with exit code 0
```

- 示例程序中，有两个线程各将3个整数写入到一个数组中，由于未能安排好写入数据的顺序，结果它们写入的数据因相互覆盖而出错。
- 每次运行的结果，可能与上次一致，也可能不一致。

未同步的多线程程序，运行结果失去了确定性，数据结果也是错误的。

示例：SharedArrayTest.java

缓存数据引发的问题-1

```
class Task {  
    //必须加入volatile, 否则程序不会结束  
    private volatile boolean isDone = false;  
    public boolean isDone() {  
        return isDone;  
    }  
    public void setDone(boolean done) {  
        isDone = done;  
    }  
}
```

VisitCachedData.java

```
class WorkerThread extends Thread {  
    Task currentTask = null;  
  
    public WorkerThread(Task task) {  
        currentTask = task;  
    }  
  
    @Override  
    public void run() {  
        for (int i = 0; i < 10; i++) {  
            try {  
                TimeUnit.SECONDS.sleep(1);  
                System.out.print(".");  
            } catch (InterruptedException e) {  
                e.printStackTrace();  
            }  
        }  
        currentTask.setDone(true);  
    }  
}
```


缓存数据问题-2

```
public class VisitCachedData {  
    public static void main(String[] args) {  
        var task = new Task();  
        new WorkerThread(task).start();  
        new Thread(() -> {  
            while (!task.isDone()) {  
            }  
            System.out.println("\n后台任务结束! ");  
        }).start();  
    }  
}
```

Task对象被两个线程所共享，一个线程负责设置值，另一个则不断地检测其值。

运行: VisitCachedData x

"C:\Program Files\Java\jdk-17\bin\java.exe"

.....

后台任务结束!

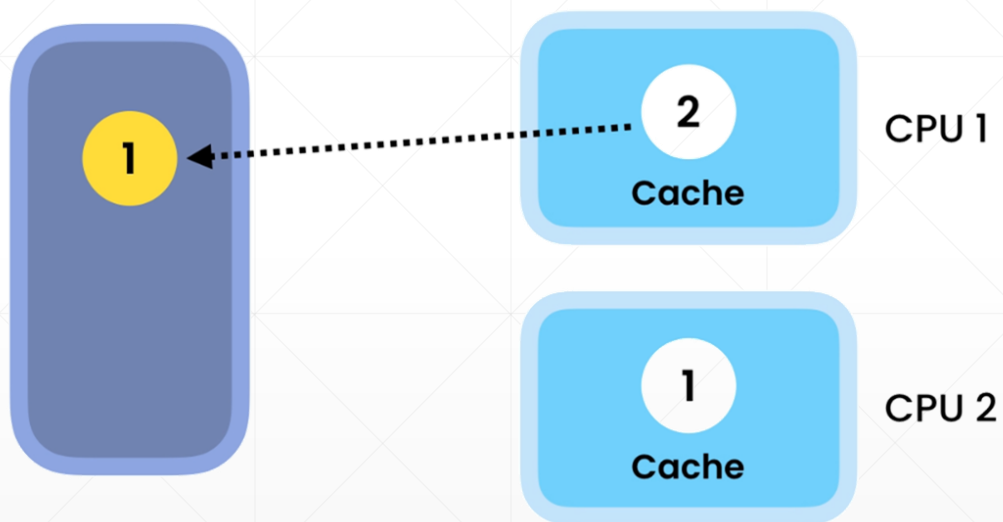
进程已结束,退出代码0



如果将Task类中isDone变量前的volatile修饰符去掉，你会发现，此程序将永远不会结束.....

原因

CPU会缓存它使用的数据，这时，缓存中的值，可能会与内存中的值不一致，从而引发问题.....



多核CPU各有各的缓存，这些缓存可能存在着数据不一致的问题。

前面示例给字段加上volatile，可以通知JVM，不要缓存这个值，从而避免出现缓存不一致的问题。

多线程开发的常见陷阱



当一段程序中存在多个线程同时访问同一个共享资源时，“**竞态 (Race Condition)**”随之产生。



修改可见性问题：当一个线程尝试修改一个共享变量的值时，由于该线程对这个共享变量的修改可能仅在当前工作线程内部的工作内存中生效，因此其他工作线程无法获知当前工作线程对这一共享变量值的修改（或者说其他线程读取到的是一个旧的无效的数据）。

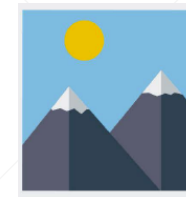


缓存一致性问题：为了提升性能，计算机软硬件系统可能会缓存数据，当多个线程同时读写不同层次的数据缓存时，有可能会带来数据不一致的问题。



为了消除多线程程序的不确定性，保证程序的正确性，我们必须采用一些方法对线程的执行进行干预，也就是要对线程的推进顺序进行同步……

线程同步



指采用某种方式实现的多个线程之间的相互协作关系，主要分为以下两类：

控制同步：比如A线程等待B线程的完成

数据同步：比如，多线程存取同一个数据，应该互斥写，但可以同时读。

后面的课程，将围绕着这两种同步方式而展开。